Master These 75 Patterns

Before Your Next Product-Based Company Interview

If you want to crack product-based companies, you don't need 1000 random questions.
You need to master patterns.

Below are 75 important DSA patterns with:
• What the pattern means
• How to identify it
• One sample question

ARRAY PATTERNS

1. Two Sum Pattern
   Identify: Find two numbers satisfying a condition.
   Sample: Two Sum

2. Prefix Sum
   Identify: Repeated range sum queries.
   Sample: Subarray Sum Equals K

3. Kadane's Algorithm
   Identify: Maximum subarray sum.
   Sample: Maximum Subarray

4. Majority Element (Voting Algorithm)
   Identify: Element appearing more than n/2 times.
   Sample: Majority Element

5. Dutch National Flag
   Identify: Sorting 0, 1, 2 in one pass.
   Sample: Sort Colors

6. Missing Number (XOR / Sum Trick)
   Identify: One number missing in 0 to n range.
   Sample: Missing Number

7. Cyclic Sort
   Identify: Numbers from 1 to n misplaced.
   Sample: First Missing Positive

8. Sliding Window (Fixed Size)
   Identify: Subarray of size k.
   Sample: Maximum Sum Subarray of Size K

9. Sliding Window (Variable Size)
   Identify: Longest/shortest subarray satisfying a condition.
   Sample: Longest Substring Without Repeating Characters

10. Merge Intervals
    Identify: Overlapping intervals.
    Sample: Merge Intervals

STRING PATTERNS

11. Anagram Check
    Identify: Same character frequency.
    Sample: Valid Anagram

12. Frequency Counter
    Identify: Character counting problems.
    Sample: First Unique Character

13. Expand Around Center (Palindrome)
    Identify: Longest palindromic substring.
    Sample: Longest Palindromic Substring

14. KMP Pattern Matching
    Identify: Efficient substring search.
    Sample: Implement strStr()

15. Rabin-Karp (Rolling Hash)
    Identify: Pattern search using hashing.
    Sample: Repeated String Match

16. String Compression
    Identify: Compress repeated characters.
    Sample: String Compression

17. Minimum Window Substring
    Identify: Smallest substring containing all characters.
    Sample: Minimum Window Substring

18. Valid Parentheses (Stack)
    Identify: Balanced brackets.
    Sample: Valid Parentheses

19. Two Pointer Palindrome Check
    Identify: Palindrome validation.
    Sample: Valid Palindrome

20. Group Anagrams
    Identify: Group strings with same sorted pattern.
    Sample: Group Anagrams

TWO POINTER PATTERNS

21. Opposite Direction Two Pointers
    Identify: Sorted array pair problems.
    Sample: Two Sum II

22. Same Direction Fast & Slow
    Identify: Remove duplicates / detect cycle.
    Sample: Remove Duplicates from Sorted Array

23. Floyd’s Cycle Detection
    Identify: Detect loop in linked list.
    Sample: Linked List Cycle

24. Trapping Rain Water
    Identify: Water accumulation problem.
    Sample: Trapping Rain Water
25. Container With Most Water
    Identify: Maximize area between two lines.
    Sample: Container With Most Water

LINKED LIST PATTERNS

26. Reverse Linked List
27. Reverse in K Groups
28. Detect and Remove Cycle
29. Find Middle Node
30. Merge Two Sorted Lists
31. LRU Cache
32. Add Two Numbers
33. Flatten Linked List

STACK PATTERNS

34. Next Greater Element
35. Monotonic Stack
36. Largest Rectangle in Histogram
37. Stock Span Problem
38. Min Stack
39. Infix to Postfix Conversion

QUEUE PATTERNS

40. Sliding Window Maximum
41. Circular Queue
42. BFS using Queue
43. Design Hit Counter
44. Rotten Oranges

BINARY SEARCH PATTERNS

45. Classic Binary Search
46. Lower Bound / Upper Bound

47. Search in Rotated Sorted Array
48. Find Peak Element
49. Binary Search on Answer (Optimization Problems)

RECURSION & BACKTRACKING

50. Subsets
51. Permutations
52. Combination Sum
53. N-Queens
54. Sudoku Solver
55. Palindrome Partitioning

TREE PATTERNS

56. DFS (Preorder / Inorder / Postorder)
57. BFS Level Order Traversal
58. Height / Diameter of Tree
59. Lowest Common Ancestor
60. Validate BST
61. Kth Smallest in BST
62. Serialize and Deserialize Tree
63. Path Sum Problems

HEAP PATTERNS

64. Top K Elements
65. Kth Largest Element
66. Merge K Sorted Lists
67. Median of Data Stream

GRAPH PATTERNS

68. DFS Traversal
69. BFS Traversal
70. Cycle Detection in Graph
71. Topological Sort
72. Dijkstra’s Algorithm

73. Union Find (Disjoint Set)
74. Bipartite Graph Check

DYNAMIC PROGRAMMING

75. 0/1 Knapsack
76. Longest Increasing Subsequence
77. Longest Common Subsequence
78. Coin Change
79. Matrix DP (Grid Problems)
80. Partition DP

How to Identify Patterns Quickly

Ask these 5 questions while solving:

1. Is it subarray or substring? → Sliding Window
2. Is array sorted? → Two Pointers / Binary Search
3. Is it a tree with levels? → BFS
4. Is it asking all combinations? → Backtracking
5. Is it optimization with choices? → DP